

Unidad 3 - Sesión 2 (Anexo): Introducción a LangGraph

Anexo a [teoria.md](#): Esta extensión profundiza en **LangGraph**, una librería del ecosistema LangChain para construir aplicaciones con LLMs como **grafos de estado**. Se asume que el alumno ha leído los Bloques 1-5 de la teoría principal (acceso programático, LangChain básico, Chat Models, Prompt Templates, Chains y Memory).

Objetivos del Anexo

Al finalizar este material, el estudiante será capaz de:

- Comprender las limitaciones del operador pipe (|) de LCEL para flujos no lineales
- Explicar el modelo mental de LangGraph: estado compartido + nodos + aristas
- Construir un grafo básico con **StateGraph**, definiendo el estado con **TypedDict** y **add_messages**
- Implementar **edges condicionales** para enrutar la ejecución según el estado
- Añadir **memoria persistente** mediante checkpoints (**MemorySaver**)
- Crear un agente ReAct mínimo con **create_react_agent** y herramientas personalizadas
- Identificar cuándo conviene usar LangGraph frente a una Chain LCEL clásica

Bloque 1: De Chains Lineales a Grafos de Estado (15 minutos)

1.1 La Limitación del Pipe |

En la teoría principal vimos que LangChain permite componer operaciones con el operador pipe:

```
cadena = prompt | modelo | parser
```

Este patrón funciona perfectamente cuando el flujo es **lineal y unidireccional**: $A \rightarrow B \rightarrow C$. Pero las aplicaciones reales con LLMs rara vez son lineales.

PROBLEMAS CON CHAINS LINEALES:

Caso 1: ENRUTAMIENTO CONDICIONAL

¿La pregunta es técnica o emocional?

└─ Técnica → Chain especializada A

└─ Emocional → Chain especializada B

Con pipe `|` no puedes expresar "if/else" entre nodos.

Caso 2: BUCLES (LOOPS)

Agente: Pensar → Usar tool → Observar → Pensar → ...
hasta que tenga la respuesta final.

Con pipe `|` no puedes volver a un nodo anterior.

Caso 3: ESTADO COMPARTIDO COMPLEJO

Múltiples nodos leen y escriben en variables comunes:
historial, documentos recuperados, resultados parciales

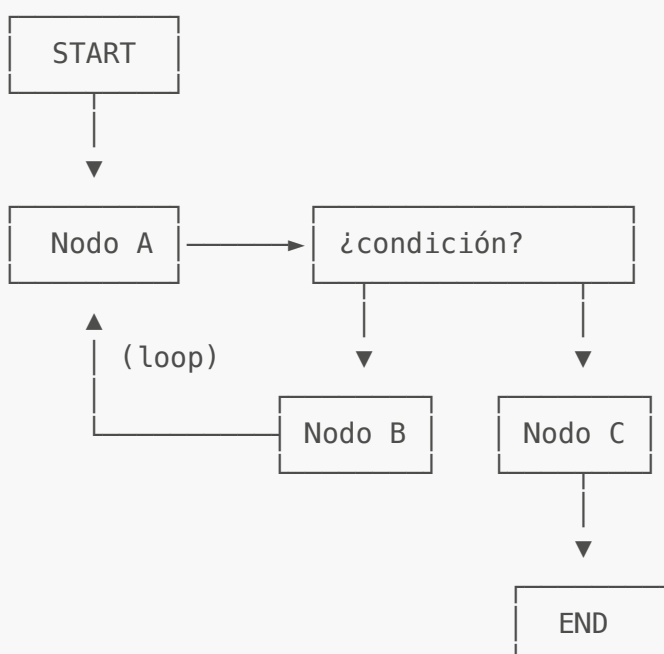
Con pipe `|` cada paso solo ve el output del anterior.

1.2 ¿Qué es LangGraph?

LangGraph es una librería construida sobre LangChain que permite expresar aplicaciones con LLMs como **grafos dirigidos**:

- **Nodos**: funciones que ejecutan una unidad de trabajo (llamar al LLM, ejecutar una tool, transformar datos...)
- **Aristas (edges)**: conexiones entre nodos, posiblemente **condicionales**
- **Estado (state)**: estructura de datos compartida que cada nodo puede leer y modificar
- **Checkpoints**: capacidad de persistir el estado para retomar conversaciones o pausar/reanudar la ejecución

MODELO MENTAL DE LangGraph:



Estado compartido: dict que viaja por todos los nodos.
Cada nodo recibe el estado y devuelve actualizaciones parciales.

1.3 Cuándo usar LangGraph vs Chain LCEL

Escenario	Recomendación
Pipeline lineal de 2-3 pasos	Chain LCEL (prompt modelo parser)
Necesidad de bifurcaciones (if/else)	LangGraph
Bucles iterativos (agentes ReAct)	LangGraph
Estado compartido entre múltiples pasos	LangGraph
Persistencia de conversaciones / human-in-the-loop	LangGraph (checkpointers)
Múltiples agentes coordinados (multi-agente)	LangGraph
Prototipo rápido sin lógica condicional	Chain LCEL

Regla práctica: si te ves dibujando flechas en un papel para entender tu aplicación, probablemente quieras LangGraph.

Bloque 2: Conceptos Fundamentales (20 minutos)

2.1 Instalación

```
pip install langgraph langchain-openai
```

LangGraph se instala como paquete separado, pero usa los mismos ChatModel que ya conoces de LangChain.

2.2 El Estado (State)

El estado es un **diccionario tipado** (normalmente un TypedDict) que define qué información viaja por el grafo. Cada clave puede tener un **reducer**: una función que decide cómo se combinan los valores nuevos con los antiguos.

```
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages

class EstadoChat(TypedDict):
    # Cuando un nodo devuelva nuevos mensajes, se AÑADIRÁN
    # a la lista existente, no la sobrescribirán.
    messages: Annotated[list, add_messages]
```

```
# Sin reducer: el valor nuevo SOBREScribe al anterior.
contador: int
```

El reducer `add_messages`

`add_messages` es el reducer más usado: convierte automáticamente diccionarios `{"role": "user", "content": "..."} a objetos HumanMessage/AIMessage y los añade al historial. Sin este reducer, cada nodo borraría los mensajes anteriores.`

SIN reducer: <pre>[m1] + nodo devuelve [m2] = [m2] (sobrescribe)</pre>	CON <code>add_messages</code>: <pre>[m1] + nodo devuelve [m2] = [m1, m2] (concatena)</pre>
--	--

Atajo: `MessagesState`

Para chatbots simples, LangGraph trae un estado predefinido:

```
from langgraph.graph import MessagesState
# Equivale a:
# class MessagesState(TypedDict):
#     messages: Annotated[list, add_messages]
```

2.3 Nodos (Nodes)

Un **nodo** es una función Python que recibe el estado actual y devuelve un diccionario con las actualizaciones. **No** devuelve el estado completo, solo los campos que cambian.

```
def mi_nodo(state: EstadoChat) -> dict:
    ultimo_mensaje = state["messages"][-1].content
    # ... hacer algo ...
    return {"messages": [{"role": "assistant", "content": "respuesta"}]}
```

Convención: el nombre del nodo (string) es el identificador que se usa en las aristas.

2.4 Aristas (Edges)

Hay tres tipos:

1. **Edge fijo:** tras el nodo A va siempre el nodo B.

```
graph.add_edge("A", "B")
```

2. **Edge condicional:** una función decide a qué nodo ir según el estado.

```
graph.add_conditional_edges("A", funcion_router)
```

3. **START** y **END**: nodos especiales que marcan el principio y fin del grafo.

```
from langgraph.graph import START, END
graph.add_edge(START, "A")
graph.add_edge("B", END)
```

2.5 Construcción y Compilación

El flujo siempre sigue tres pasos: **definir** → **compilar** → **invocar**.

```
from langgraph.graph import StateGraph, START, END

# 1) Definir el grafo
builder = StateGraph(EstadoChat)
builder.add_node("nodo_a", funcion_a)
builder.add_node("nodo_b", funcion_b)
builder.add_edge(START, "nodo_a")
builder.add_edge("nodo_a", "nodo_b")
builder.add_edge("nodo_b", END)

# 2) Compilar (crea el "programa" ejecutable)
graph = builder.compile()

# 3) Invocar
resultado = graph.invoke({"messages": [{"role": "user", "content": "Hola"}]})
print(resultado["messages"][-1].content)
```

Bloque 3: Primer Grafo - Chatbot con LangGraph (20 minutos)

3.1 Versión Mínima: Un Solo Nodo

Empecemos por el equivalente a **prompt** | **modelo** pero como grafo:

```
import os
from dotenv import load_dotenv
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI

load_dotenv()

# Modelo (puedes usar OpenAI, OpenRouter, Claude, etc.)
modelo = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
```

```
# Nodo: recibe el estado, llama al modelo, devuelve la respuesta
def llamar_modelo(state: MessagesState) -> dict:
    respuesta = modelo.invoke(state["messages"])
    return {"messages": [respuesta]}

# Construir el grafo
builder = StateGraph(MessagesState)
builder.add_node("chatbot", llamar_modelo)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

graph = builder.compile()

# Invocar
resultado = graph.invoke({
    "messages": [{"role": "user", "content": "¿Qué es Python?"}]
})

# La última posición del historial es la respuesta del modelo
print(resultado["messages"][-1].content)
```

DIAGRAMA DEL GRAFO ANTERIOR:

START → chatbot → END

Estado: { messages: [HumanMessage, AIMessage] }

3.2 Visualizar el Grafo

LangGraph permite generar un diagrama del grafo (útil para depurar):

```
# En Jupyter / Colab
from IPython.display import Image, display

display(Image(graph.get_graph().draw_mermaid_png()))

# O simplemente imprimir el ASCII
print(graph.get_graph().draw_ascii())
```

3.3 ¿Qué Hemos Ganado vs prompt | modelo?

A primera vista parece más código. Pero el grafo nos da:

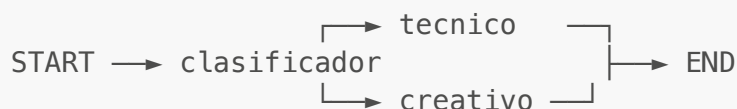
- **Estado explícito:** `state["messages"]` está siempre disponible
- **Punto de extensión:** añadir un nodo nuevo (validador, logger, herramienta) es trivial
- **Bucles e if/else:** imposibles con |

- **Persistencia gratuita:** con un checkpointer (Bloque 5) tenemos memoria de conversación sin escribir código de gestión

Bloque 4: Edges Condicionales (20 minutos)

4.1 El Patrón "Router"

Imaginemos un asistente que clasifica preguntas y las dirige a un experto distinto según el tema:



4.2 Implementación

```

from typing import Annotated, TypedDict, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_openai import ChatOpenAI

class EstadoRouter(TypedDict):
    messages: Annotated[list, add_messages]
    categoria: str # "tecnico" o "creativo"

modelo = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# --- Nodo 1: clasificar la pregunta ---
def clasificar(state: EstadoRouter) -> dict:
    pregunta = state["messages"][-1].content
    prompt = (
        f"Clasifica la siguiente pregunta como 'tecnico' o 'creativo'. "
        f"Responde SÓLO con una de las dos palabras.\n\nPregunta: "
        f"{pregunta}"
    )
    respuesta = modelo.invoke(prompt).content.strip().lower()
    categoria = "tecnico" if "tecnico" in respuesta else "creativo"
    return {"categoria": categoria}

# --- Nodo 2a: experto técnico ---
def experto_tecnico(state: EstadoRouter) -> dict:
    pregunta = state["messages"][-1].content
    respuesta = modelo.invoke(
        f"Eres un ingeniero senior. Responde con precisión "
        f"técnica:\n{pregunta}"
    )
    return {"messages": [respuesta]}

# --- Nodo 2b: experto creativo ---
def experto_creativo(state: EstadoRouter) -> dict:

```

```

pregunta = state["messages"][-1].content
respuesta = modelo.invoke(
    f"Eres un escritor creativo. Responde con metáforas e
    imaginación:\n{pregunta}"
)
return {"messages": [respuesta]}

# --- Función de enrutado ---
def decidir_ruta(state: EstadoRouter) -> Literal["experto_tecnico",
"experto_creativo"]:
    """Devuelve el nombre del siguiente nodo según el estado."""
    if state["categoria"] == "tecnico":
        return "experto_tecnico"
    return "experto_creativo"

# --- Construir el grafo ---
builder = StateGraph(EstadoRouter)
builder.add_node("clasificar", clasificar)
builder.add_node("experto_tecnico", experto_tecnico)
builder.add_node("experto_creativo", experto_creativo)

builder.add_edge(START, "clasificar")

# Edge condicional: tras "clasificar" llamamos a decidir_ruta
# para saber a qué nodo ir.
builder.add_conditional_edges("clasificar", decidir_ruta)

builder.add_edge("experto_tecnico", END)
builder.add_edge("experto_creativo", END)

graph = builder.compile()

# Probar
r1 = graph.invoke({"messages": [{"role": "user", "content": "¿Cómo funciona
TCP/IP?"}]})
r2 = graph.invoke({"messages": [{"role": "user", "content": "Inventa una
historia sobre un dragón"}]})
print("TÉCNICA ->", r1["messages"][-1].content[:150], "...")
print("CREATIVA ->", r2["messages"][-1].content[:150], "...")

```

4.3 Conditional Edges con Mapeo Explícito

A veces queremos que la función router devuelva **claves** que mapean a nombres de nodo. Útil cuando los nombres son largos o cambian:

```

def decidir_ruta(state) -> str:
    return "tech" if state["categoria"] == "tecnico" else "creative"

builder.add_conditional_edges(
    "clasificar",
    decidir_ruta,

```



```
{
    "tech": "experto_tecnico",
    "creative": "experto_creativo",
},
)
```

Bloque 5: Memoria con Checkpointers (15 minutos)

5.1 El Problema de la Memoria

En la teoría principal (sección 4.1) implementamos la memoria del chatbot **manualmente**: una lista que crecía con cada turno. LangGraph automatiza esto mediante **checkpointers**.

Un checkpointer guarda el estado del grafo después de cada nodo, asociado a un `thread_id`. Si invocamos el grafo de nuevo con el mismo `thread_id`, recupera el estado previo automáticamente.

5.2 `MemorySaver`: Checkpointer en RAM

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, MessagesState, START, END
from langchain_openai import ChatOpenAI

modelo = ChatOpenAI(model="gpt-4o-mini")

def llamar_modelo(state: MessagesState) -> dict:
    return {"messages": [modelo.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chatbot", llamar_modelo)
builder.add_edge(START, "chatbot")
builder.add_edge("chatbot", END)

# Compilar CON checkpointer
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

# Configuración de la sesión
config = {"configurable": {"thread_id": "usuario_42"}}

# Turno 1
graph.invoke(
    {"messages": [{"role": "user", "content": "Hola, me llamo Ana"}]},
    config=config,
)

# Turno 2: el grafo recuerda el contexto gracias al thread_id
respuesta = graph.invoke(
    {"messages": [{"role": "user", "content": "¿Cómo me llamo?"}]},
    config=config,
)
```

```
print(respuesta["messages"][-1].content)
# → "Te llamas Ana."
```

FLUJO DEL CHECKPOINTER:

Turno 1 (thread_id="usuario_42"):

```
estado_inicial = {messages: [HumanMessage("Hola, me llamo Ana")]}
→ nodo chatbot → {messages: [..., AIMessage("¡Hola Ana!")]}
→ checkpointer GUARDA el estado en thread_id="usuario_42"
```

Turno 2 (thread_id="usuario_42"):

```
→ checkpointer RECUPERA el estado guardado
estado_inicial = {messages: [HumanMessage, AIMessage]} ← historia previa
→ se añade el nuevo mensaje del usuario
→ nodo chatbot ve TODO el historial
```

5.3 Otros Checkpointers

Checkpoint	Almacenamiento	Caso de uso
MemorySaver	RAM	Pruebas, prototipos
SqliteSaver	SQLite local	Desarrollo, apps de escritorio
PostgresSaver	PostgreSQL	Producción, multi-usuario

```
# Ejemplo con SQLite
from langgraph.checkpoint.sqlite import SqliteSaver

with SqliteSaver.from_conn_string("checkpoints.db") as checkpointer:
    graph = builder.compile(checkpointer=checkpointer)
    # ... usar el grafo ...
```

5.4 Inspeccionar el Historial

Con un checkpointer, podemos ver el estado actual o recorrer la historia de un thread:

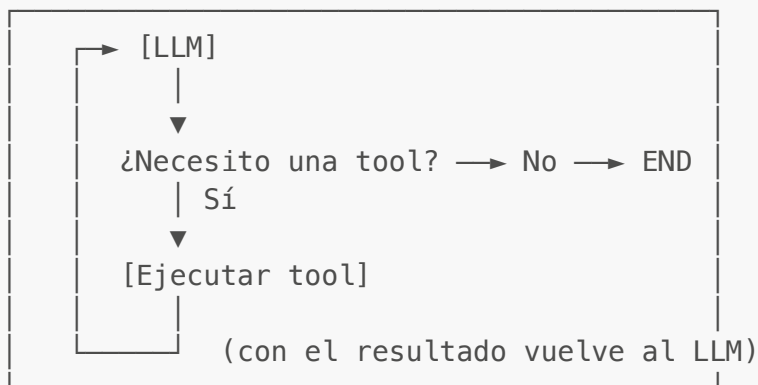
```
# Estado actual
estado = graph.get_state(config)
print(f"Mensajes guardados: {len(estado.values['messages'])}")

# Historial completo (lista de checkpoints)
for snapshot in graph.get_state_history(config):
    print(snapshot.values["messages"][-1].content[:80])
```

Bloque 6: Agentes con `create_react_agent` (15 minutos)

6.1 Patrón ReAct (Reasoning + Acting)

Un agente ReAct itera sobre este bucle hasta tener suficiente información:



LangGraph lo construye en una línea con `create_react_agent`. Esto será la base de la **Unidad 4 (Agentes)**.

6.2 Ejemplo: Agente con una Calculadora

```

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from langgraph.prebuilt import create_react_agent
from langgraph.checkpoint.memory import MemorySaver

# 1) Definir herramientas con el decorador @tool
@tool
def multiplicar(a: float, b: float) -> float:
    """Multiplica dos números y devuelve el resultado."""
    return a * b

@tool
def obtener_clima(ciudad: str) -> str:
    """Devuelve el clima actual de una ciudad (datos simulados)."""
    datos = {
        "Madrid": "soleado, 22°C",
        "Bilbao": "lluvioso, 14°C",
        "Sevilla": "soleado, 28°C",
    }
    return datos.get(ciudad, "datos no disponibles")

tools = [multiplicar, obtener_clima]

# 2) Crear el agente
modelo = ChatOpenAI(model="gpt-4o-mini")
agente = create_react_agent(
    modelo,
  
```

```

tools=tools,
checkpointer=MemorySaver(), # opcional, para memoria de conversación
)

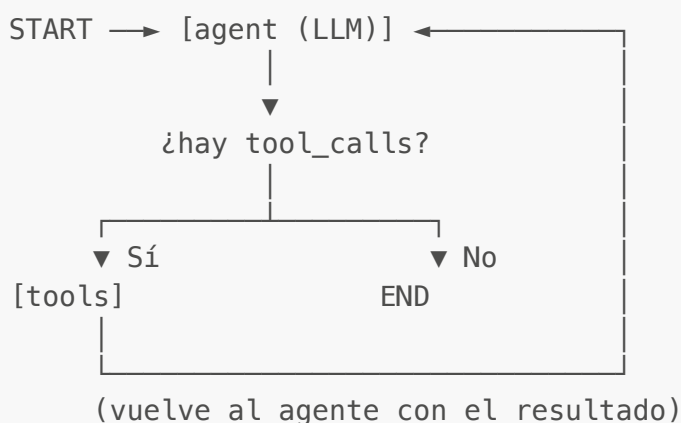
# 3) Invocar
config = {"configurable": {"thread_id": "demo"}}
respuesta = agente.invoke(
    {"messages": [{"role": "user",
                    "content": "¿Qué tiempo hace en Madrid? "
                    "Y dime cuánto es 17 por 23."}]},
    config=config,
)

# Imprimir todo el "razonamiento" del agente
for m in respuesta["messages"]:
    m.pretty_print()

```

6.3 Lo Que Está Pasando "Por Dentro"

`create_react_agent` construye este grafo automáticamente:



Es exactamente el patrón ReAct, expresado como grafo con un edge condicional y un bucle.

Bloque 7: Streaming y Conexión con Próximas Unidades (10 minutos)

7.1 Streaming de Eventos

Como cualquier `Runnable` de LangChain, los grafos de LangGraph soportan `.stream()`:

```

config = {"configurable": {"thread_id": "demo"}}

for chunk in agente.stream(
    {"messages": [{"role": "user", "content": "¿Tiempo en Sevilla?"}]},
    config=config,
    stream_mode="updates", # emite el delta de cada nodo
):

```

```
for nodo, actualizacion in chunk.items():
    print(f"--- Salida del nodo: {nodo} ---")
    print(actualizacion)
```

`stream_mode` puede ser `"updates"` (solo cambios), `"values"` (estado completo) o `"messages"` (token a token).

7.2 Comparativa Final

Aspecto	Chain LCEL ()	LangGraph
Curva de aprendizaje	Baja	Media
Flujos lineales	Excelente, conciso	Innecesariamente verboso
Bifurcaciones / loops	No soportado	Excelente
Memoria de conversación	Manual o <code>RunnableWithMessageHistory</code>	Checkpointing integrado
Agentes con tools	Posible pero verboso	<code>create_react_agent</code> (1 línea)
Visualización del flujo	No nativa	<code>draw_mermaid_png()</code>
Producción / escalabilidad	Buena	Muy buena (multi-thread, persistencia)

7.3 Hoja de Ruta del Curso

LANGGRAPH EN EL CURSO ML2:

Unidad 3 (esta sesión, anexo):
Conceptos básicos: State, Nodes, Edges, Checkpointers
→ "Saber leer un grafo y construir uno simple"

Unidad 4 (Agentes):
`create_react_agent` en profundidad
Tools personalizadas, multi-agente, supervisión
→ "Construir agentes que decidan qué hacer"

Unidad 5 (RAG):
LangGraph como orquestador de pipelines RAG
Nodos: retriever → re-ranker → generador
→ "RAG avanzado con loops de auto-corrección"

Unidad 6 (MCP):
Integrar herramientas MCP como nodos del grafo
→ "Tools desacopladas + LangGraph como orquestador"

Resumen del Anexo

Conceptos Clave

- 1. **LangGraph** modela aplicaciones con LLMs como **grafos de estado**: nodos (funciones), edges (transiciones, posiblemente condicionales) y un estado compartido.
- 2. El **estado** es un **TypedDict** con reducers (**add_messages** para historiales de chat).
- 3. Los **edges condicionales** permiten enrutar la ejecución según el estado, expresando **if/else** y bucles.
- 4. Los **checkpointers** (**MemorySaver**, **SqliteSaver**, **PostgresSaver**) dan memoria persistente por **thread_id** sin código manual.
- 5. **create_react_agent** construye un agente ReAct completo en una línea, sentando la base de la Unidad 4.

Qué Deberías Saber Hacer

Habilidad	Nivel Esperado
Definir un State con TypedDict y add_messages	Sin consultar documentación
Construir un grafo lineal con StateGraph , START , END	Sin consultar documentación
Añadir un edge condicional con función router	Con apoyo de un ejemplo
Compilar con MemorySaver y usar thread_id	Sin consultar documentación
Crear un agente con create_react_agent y una tool propia	Con apoyo de un ejemplo
Decidir entre Chain LCEL y LangGraph para un problema dado	Sí

Conexión con la Práctica

Los ejercicios guiados de este anexo están en [ejercicios2.md](#). Construirás de forma incremental:

- 1. Un chatbot mínimo con **StateGraph**
- 2. El mismo chatbot con memoria (**MemorySaver**)
- 3. Un grafo con edge condicional (router)
- 4. Un agente ReAct con una tool personalizada

Referencias

- LangGraph. (2026). Documentation. <https://langchain-ai.github.io/langgraph/>
- LangGraph. (2026). Tutorials - Quickstart. <https://langchain-ai.github.io/langgraph/tutorials/introduction/>
- LangGraph. (2026). Conceptual guide - Low Level. https://langchain-ai.github.io/langgraph/concepts/low_level/
- LangChain. (2026). **create_react_agent** API Reference. <https://langchain-ai.github.io/langgraph/reference/prebuilt/>
- Yao, S., Zhao, J., Yu, D., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. <https://arxiv.org/abs/2210.03629>